Trabalho final: arquivos, exceções e revisão com IA

Atividade final de Linguagem de Programação I. Evolui o sistema da Unidade 2, no mesmo repositório.

1. Objetivo

Acrescentar três recursos ao sistema que você construiu na Unidade 2. Como nas unidades anteriores, **a IA escreve o código e você orienta**: decide o que pedir, confere o resultado e responde por ele. Cada recurso trabalha uma habilidade diferente:

- **Persistência em arquivo.** O sistema guarda os dados entre uma execução e outra. Aplica a aula de arquivos a um sistema seu.
- **Tratamento de erros com exceções.** O sistema reage a entradas inválidas e a falhas de arquivo sem encerrar. Substitui o `bool` como forma de sinalizar erro e separa o fluxo normal do fluxo de erro.
- **Revisão e testes com IA.** A IA lê o código pronto, aponta fragilidades e sugere cenários de teste, e você julga o que procede. Completa o ciclo da disciplina: pedir código, planejar código e conferir código.

A IA escreve o código, mas entender o que ela escreveu faz parte do trabalho. Depois de cada pedido você abre o que ela gerou, confere se ela fez o que você pediu e registra o que encontrou no `README.md`, na seção "Evolução".

Quem não terminou a Unidade 2 deve concluí-la antes, no mesmo repositório.

A entrega é o link do repositório público, enviado numa tarefa do SIGAA. O prazo fica registrado na tarefa.

2. Como trabalhar

Valem as práticas da seção 5 da Unidade 2: um pedido por vez, plano antes de editar, compilar e conferir você mesmo. O código entregue é seu. Por isso cada passo termina com um momento de **olhar o que a IA fez**: você lê o código gerado (`git diff`) e escreve na seção "Evolução" do `README.md`, com as suas palavras, o que encontrou. Não precisa ser longo: duas ou três frases.

Modelo. Use o OpenCode, como nas unidades anteriores. Além do Big Pickle, hoje há outros modelos gratuitos, com `-free` no nome, como `glm-5.3-flash-free` e `deepseek-v4.1-flash-free`. Pode experimentar, e vale comparar: a revisão do passo 3 costuma mostrar diferenças entre modelos. Os modelos gratuitos mudam com frequência, e um deles pode sair ou deixar de ser gratuito; se um não aparecer, escolha outro. Para trocar, digite `/models` no OpenCode e escolha um marcado como gratuito. Anote no `README.md` qual usou.

3. Passo 1: salvar e carregar em arquivo

Use o material da aula de arquivos (`06_arquivos`) para entender o que a IA vai escrever. O sistema precisa:

- ao **sair**, salvar os dados num arquivo de texto;
- ao **iniciar**, carregar os dados desse arquivo, se ele existir. Se não existir, o programa começa vazio.

Regras:

- A leitura e a escrita ficam numa **classe própria**, por exemplo `Arquivo` ou `Repositorio`, com dois métodos: `salvar` e `carregar`. As outras classes do sistema não abrem arquivo (é a mesma ideia de separar a lógica da interface, da Unidade 2).
- Um registro por linha, com os campos separados por `;`. Num sistema de livros, por exemplo:

```
O Hobbit;Tolkien;9
Dom Casmurro;Machado de Assis;8
```

Adapte ao seu tema: o que importa é uma linha por objeto e um separador que não apareça nos dados. Se o seu sistema tem objetos que se referem a outros (uma consulta que liga paciente e médico, uma partida que liga dois times), guarde o nome ou código do outro objeto, e não uma cópia dele.

- O arquivo de dados **não entra no repositório**: coloque o nome dele no `.gitignore`.

Um pedido para começar:

```
Leia @DESCRICAO.md e @PLANO.md. Não altere nenhum arquivo. Proponha uma
classe própria para salvar e carregar os dados em arquivo de texto, com o
formato de cada linha, e diga quais arquivos você criaria ou alteraria.
```

Olhe o que a IA fez. Abra os arquivos que ela criou ou alterou e confira: a classe de arquivo é separada das outras? O formato da linha é o que você pediu? O que o código faz quando o arquivo não existe? Você consegue explicar como uma linha do arquivo vira um objeto e como um objeto vira uma linha? Escreva em "Evolução" o que encontrou: se ela fez o que você pediu, o que veio diferente do esperado e o que você pediu para ajustar.

4. Passo 2: tratar erros com exceções

Esta seção apresenta o básico de exceções, que é o que você precisa para ler e julgar o código que a IA escrever.

4.1 O problema

Alguns erros não dependem de o código estar correto: o usuário digita `abc` onde se esperava um número, ou o arquivo de dados foi apagado. O programa precisa **reagir sem encerrar**.

Com `bool`, quem chamou a função precisa lembrar de conferir o resultado, e se não conferir o erro passa em silêncio. Com exceção, a função **lança** o erro, e ele sobe até alguém **pegar**. Se ninguém pegar, o programa para na hora e avisa, em vez de seguir errado.

4.2 As três palavras

- `throw` lança o erro, no ponto em que ele foi percebido.
- `try` diz "tente fazer isto".
- `catch` diz "se der erro, trate aqui".

Exemplo completo, para copiar e rodar:

```
#include <iostream>
#include <stdexcept>
using namespace std;

int dividir(int a, int b) {
    if (b == 0) {
        throw invalid_argument("divisao por zero");   // lança o erro
    }
    return a / b;
}

int main() {
    try {
        cout << dividir(10, 2) << endl;   // imprime 5
        cout << dividir(10, 0) << endl;   // lança o erro: o resto do try é pulado
        cout << "isto nunca aparece" << endl;
    } catch (const exception& e) {
        cout << "Erro: " << e.what() << endl;   // Erro: divisao por zero
    }
    cout << "o programa continua" << endl;
    return 0;
}
```

Saída:

```
5
Erro: divisao por zero
o programa continua
```

Quando o `throw` acontece, o resto do `try` é pulado e o programa vai para o `catch`. O `e.what()` devolve o texto escrito no `throw`. Depois do `catch`, o programa segue normalmente.

Use sempre o `catch` neste formato, `catch (const exception& e)`, e os dois tipos de erro que já vêm no C++ (com `#include <stdexcept>`): `invalid_argument` para um valor que não serve (nota fora do intervalo, texto vazio) e `runtime_error` para algo que falhou na hora de rodar (arquivo que não abre). Não é preciso criar tipos novos.

4.3 Dois casos que você vai usar

Número digitado errado. A função `stoi` converte texto em número, e ela mesma lança `invalid_argument` quando o texto não é um número:

```
#include <string>

int lerNumero(string texto) {
    return stoi(texto);   // "12" vira 12; "abc" lança invalid_argument
}

// no menu:
try {
    int nota = lerNumero(entrada);
    // ...usa a nota
} catch (const exception& e) {
    cout << "Digite um numero valido." << endl;
}
```

Arquivo que não abre. O `ifstream` não lança nada sozinho: quando o arquivo não existe, ele só fica "fechado". Quem lança é você:

```
#include <fstream>

void Arquivo::carregar(string nome) {
    ifstream entrada(nome);
    if (!entrada.is_open()) {
        throw runtime_error("nao consegui abrir " + nome);
    }
    // ...le as linhas
}
```

4.4 O que fazer no seu sistema

- Em **pelo menos dois lugares** onde antes você devolvia `bool` ou ignorava o problema, passe a usar `throw` (uma nota inválida, um item que não existe, um arquivo que não abre).
- Os `try`/`catch` ficam na **interface** (o menu), que é quem conversa com o usuário. As classes lançam; o menu pega e mostra a mensagem.
- Digitar texto onde se esperava número **nunca** pode derrubar o programa.

Um pedido para começar:

```
Leia @DESCRICAO.md e @PLANO.md. Não altere nenhum arquivo. Diga em quais
lugares do código uma exceção (invalid_argument ou runtime_error) substituiria
um bool ou um erro ignorado, e onde ficariam os try/catch. Só depois de eu
aprovar, implemente.
```

Olhe o que a IA fez. Procure no código cada `throw` e cada `catch`: onde o erro é lançado, onde é tratado e o que o usuário vê na tela? Os `try`/`catch` ficaram no menu, como pedido? Teste digitando texto onde se espera número. Escreva em "Evolução" o que encontrou.

5. Passo 3: a IA como revisora

5.1 Prepare o `AGENTS.md`

O `AGENTS.md` da Unidade 2 continua valendo. Acrescente no final este trecho, para a IA saber o que mudou:

```
## Trabalho final (arquivo, exceções e revisão)

- Arquivo de texto com `<fstream>` e exceções com `<stdexcept>` passam a ser
  permitidos (`invalid_argument` e `runtime_error`). Não crie tipos de
  exceção novos.
- A leitura e a escrita de arquivo ficam numa classe própria. As outras
  classes não abrem arquivo.
- `throw` nas classes do sistema; `try`/`catch` somente na interface (menu).
- Quando o estudante pedir uma revisão, não edite nenhum arquivo: liste os
  problemas com o arquivo e a linha.
- Não altere a seção "Evolução" do `README.md`. Ela é escrita pelo
  estudante.
```

5.2 Revisão, sem editar

Peça à IA para ler o projeto e apontar problemas, sem mudar nada:

```
Leia o projeto inteiro. Não altere nenhum arquivo. Aponte o que pode dar
errado: entradas inválidas, casos em que o programa quebra, regras da
@DESCRICAO.md que o código não cumpre. Liste em ordem de importância,
com o arquivo e a linha.
```

Leia a lista com atenção e confira cada item no código. A IA erra também: às vezes aponta como defeito algo que está certo. **Você decide** o que é problema de verdade.

5.3 Cenários de teste

Peça à IA arquivos de entrada para testar o que mudou, no formato que você já usa em `exemplos/`:

```
Crie cenários em exemplos/ para testar o sistema: um com entradas inválidas
(texto onde se espera número) e um que roda sem arquivo de dados. Não altere
o código do sistema.
```

Rode cada um e confira se o resultado é o que você esperava. Se não for, ou o sistema tem um defeito, ou o cenário está errado. Descobrir qual dos dois é justamente o exercício.

5.4 Corrija o que você aceitou

Escolha pelo menos **dois** problemas da revisão ou dos testes e corrija, com um commit por correção. Use a IA para corrigir, um pedido por vez, como na Unidade 2.

6. O que entregar

Envie o link do repositório público na tarefa do SIGAA, **até a data que estiver na tarefa**.

O repositório é o da Unidade 2, com os novos commits:

- a classe de arquivo, salvando e carregando;
- exceções em pelo menos dois lugares, com `try`/`catch` no menu;
- os cenários em `exemplos/`;
- o trecho novo no `AGENTS.md`;
- a seção **"Evolução"** no `README.md`, escrita por você, sem IA (modelo abaixo);
- nenhum executável, `.o` ou arquivo de dados commitado.

Modelo da seção "Evolução", para acrescentar ao final do `README.md`. Vá preenchendo ao longo do trabalho, depois de cada passo:

```
## Evolução

### Arquivo
- O que pedi à IA:
- O que encontrei ao ler o código (ela fez o que eu pedi? algo veio
  diferente?):

### Exceções
- O que pedi à IA:
- O que encontrei ao ler o código (onde lança, onde trata, o que o usuário vê):

### Revisão
- O que a IA apontou, e os dois problemas que corrigi:
- Um problema que a IA apontou e eu não corrigi, e por quê (ou, se ela só
  acertou, uma coisa que ela errou):
```

7. Como será avaliado

- O sistema salva e carrega, e começa vazio se não há arquivo.
- Há exceções em dois lugares ou mais, e o menu não quebra com entrada ruim.
- Os cenários de teste existem e rodam.
- A seção "Evolução", com as suas palavras, mostrando que você leu o que a IA gerou e batendo com o histórico de commits.

8. Uso responsável

Use só dados fictícios. Não cole chaves, senhas ou tokens em pedidos. O código entregue é seu: você responde por ele, inclusive pelo que a IA escreveu.